

In a B+ tree, if we assume that the search key must be unique, (i.e., it doesn't appear in more than one record), what is the maximum number of times it can appear in the B+ tree?

A. none

B. once

C. twice

D. more

In a B+ tree, what is the typical maximum number of children per internal node?

A. four

B. approximately 1 thousand

C. approximately 1 million

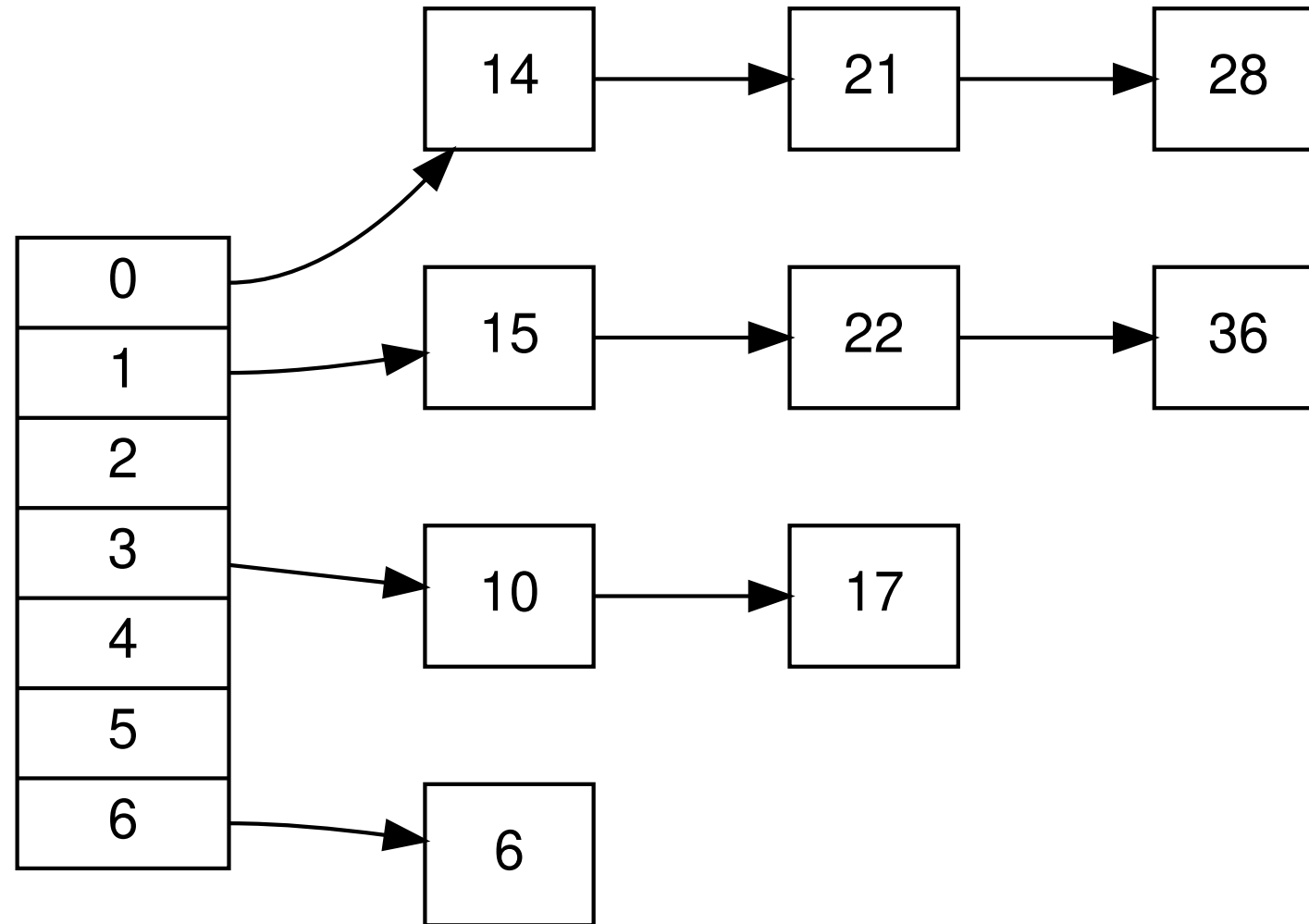
D. approximately 1 billion

In a B+ tree, if there is only one record in it, how many nodes are there?

- A. one node, which is structured as a leaf node (but also serves in practice as the root)
- B. one node, which is structured as an internal node (but also serves in practice as the one leaf)
- C. two nodes, where one is a leaf and the other is an internal node that serves as the root
- D. three nodes: one root (internal) and two leaves, since the root node must have at least two pointers

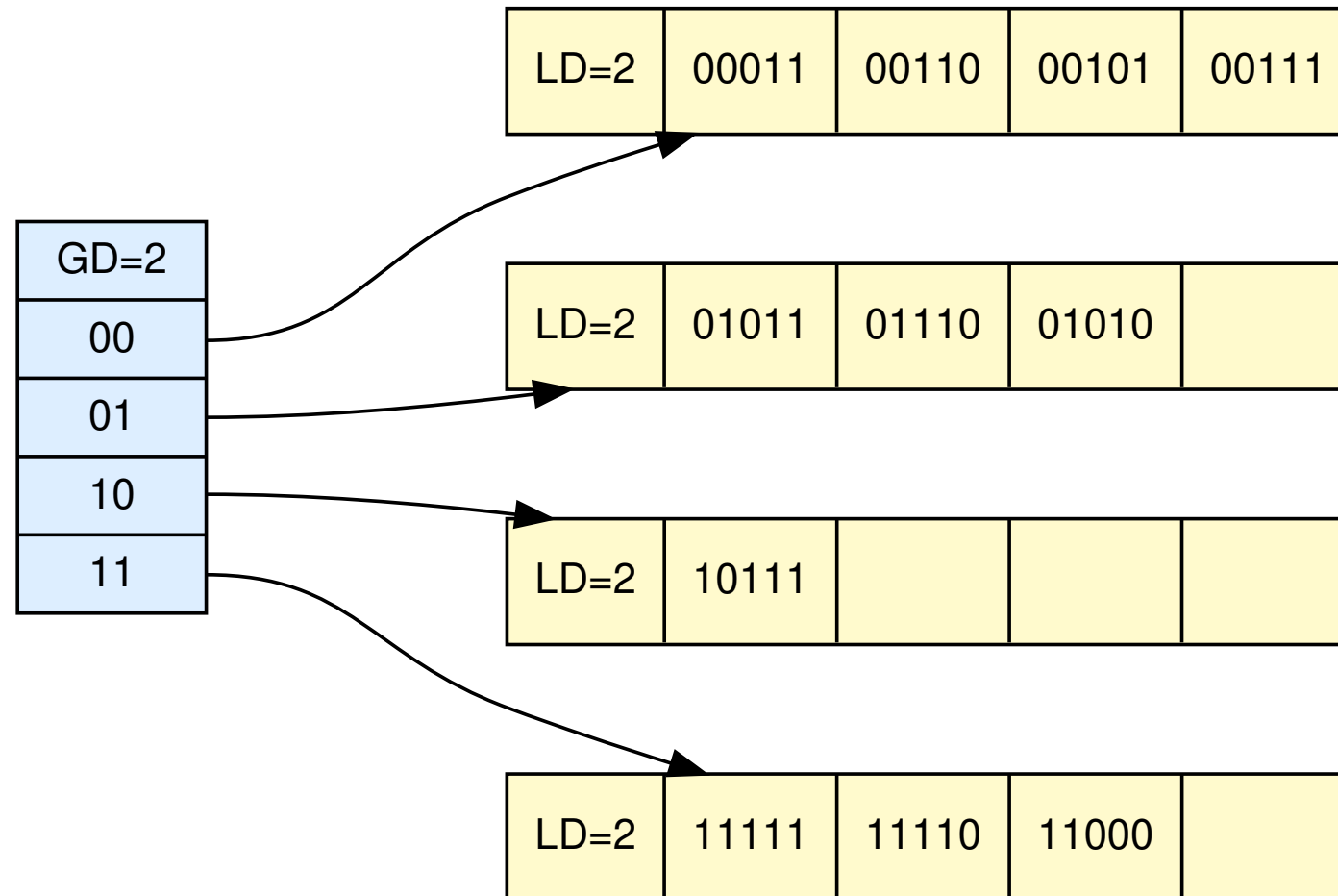
Hash based indexes

- great for equality based lookups
- terrible for range lookups



Extendible hashing! Assume a bucket capacity of **4 keys**.

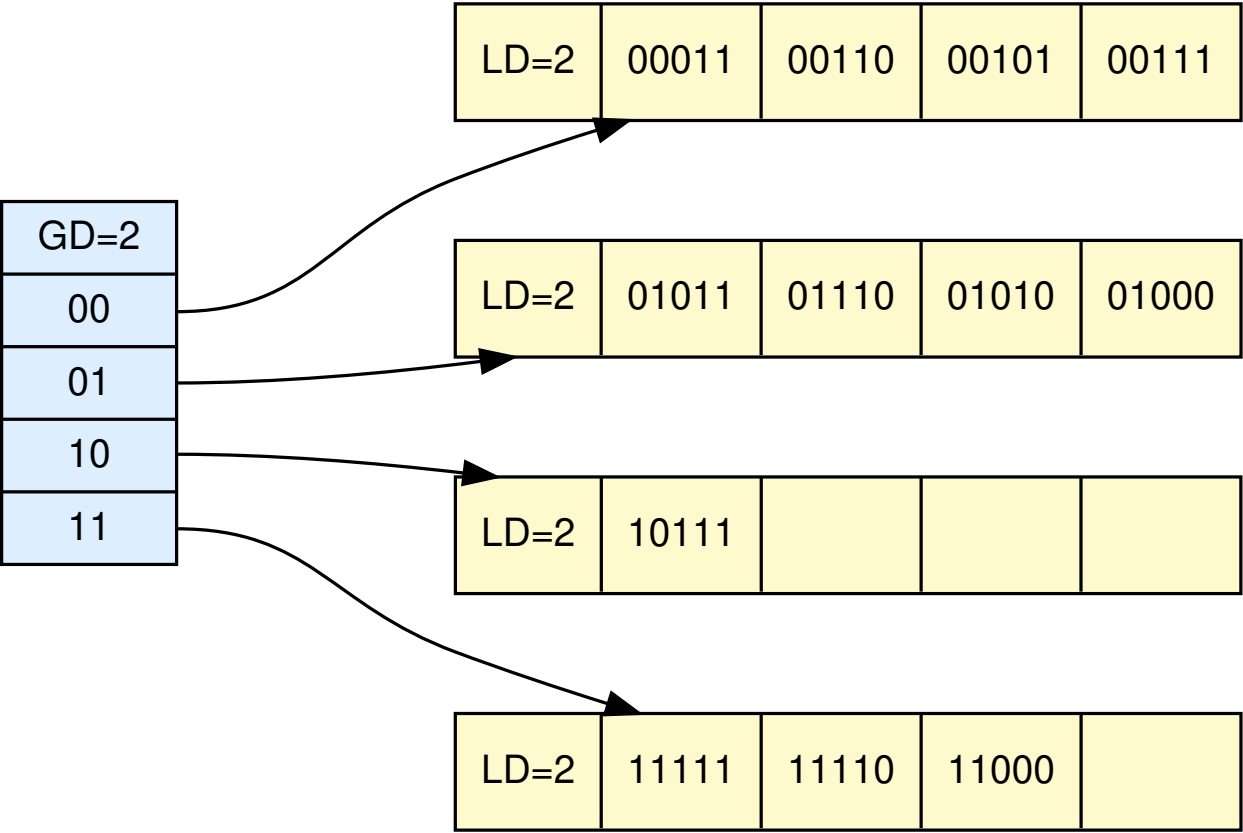
Directory shows Global Depth bits from left (most significant). Each page has a Local Depth.



Create $h(\text{key}) = \dots$
something binary and
many digits

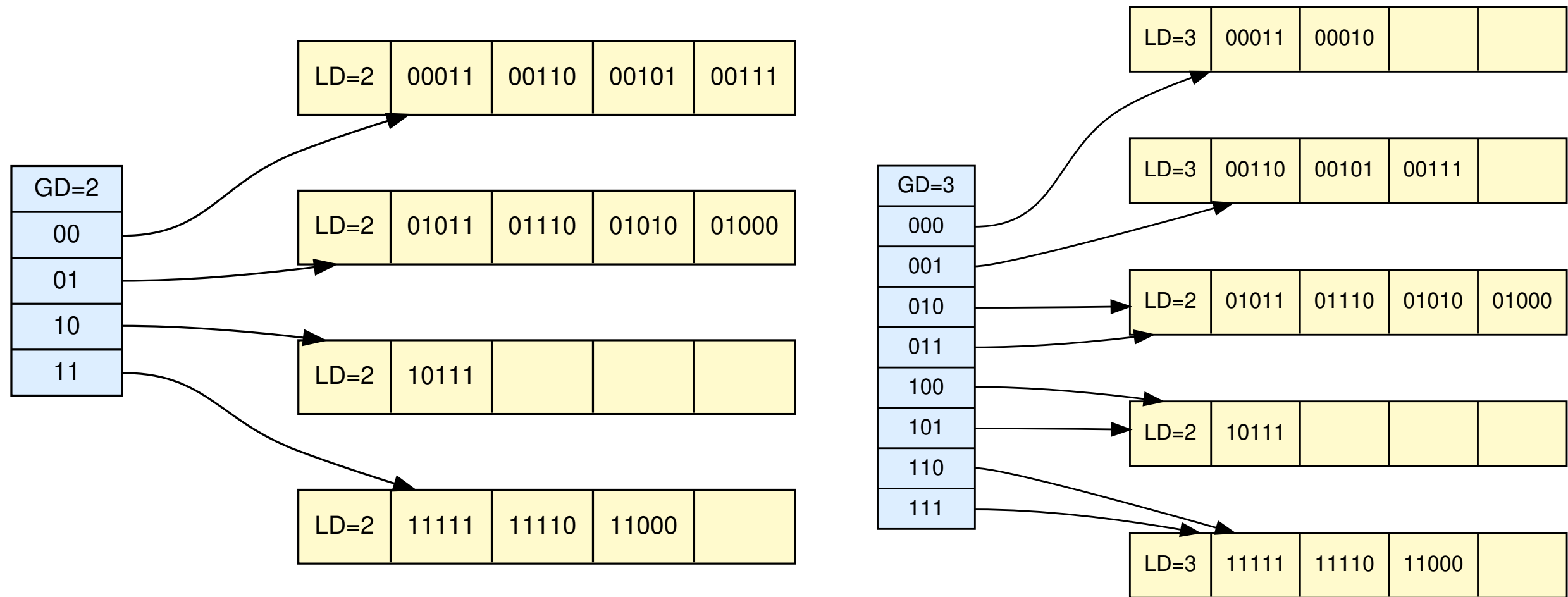
Just like B+ trees,
each entry also has an
RID to find tuple

Add a 01000.



Add a 00010. No room. Directory doubles. Bucket splits.

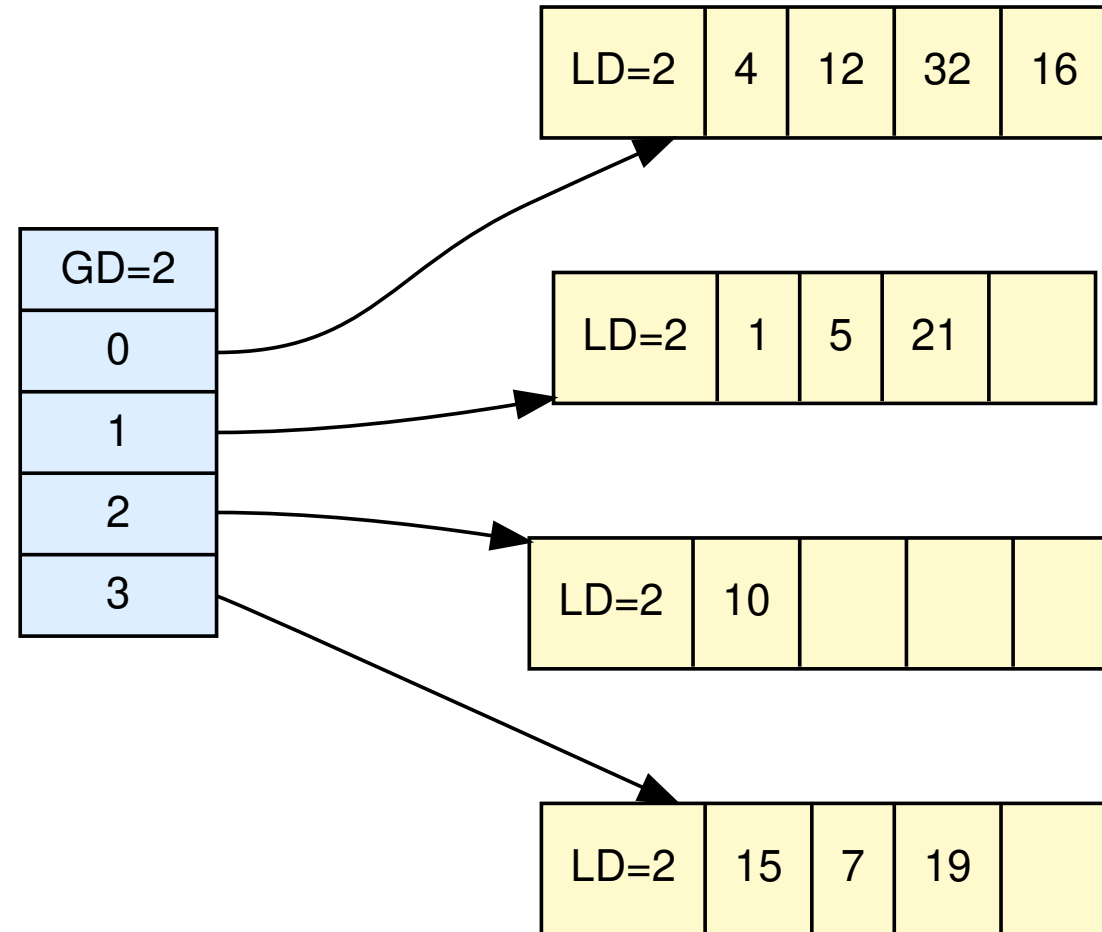
Add a 01000.



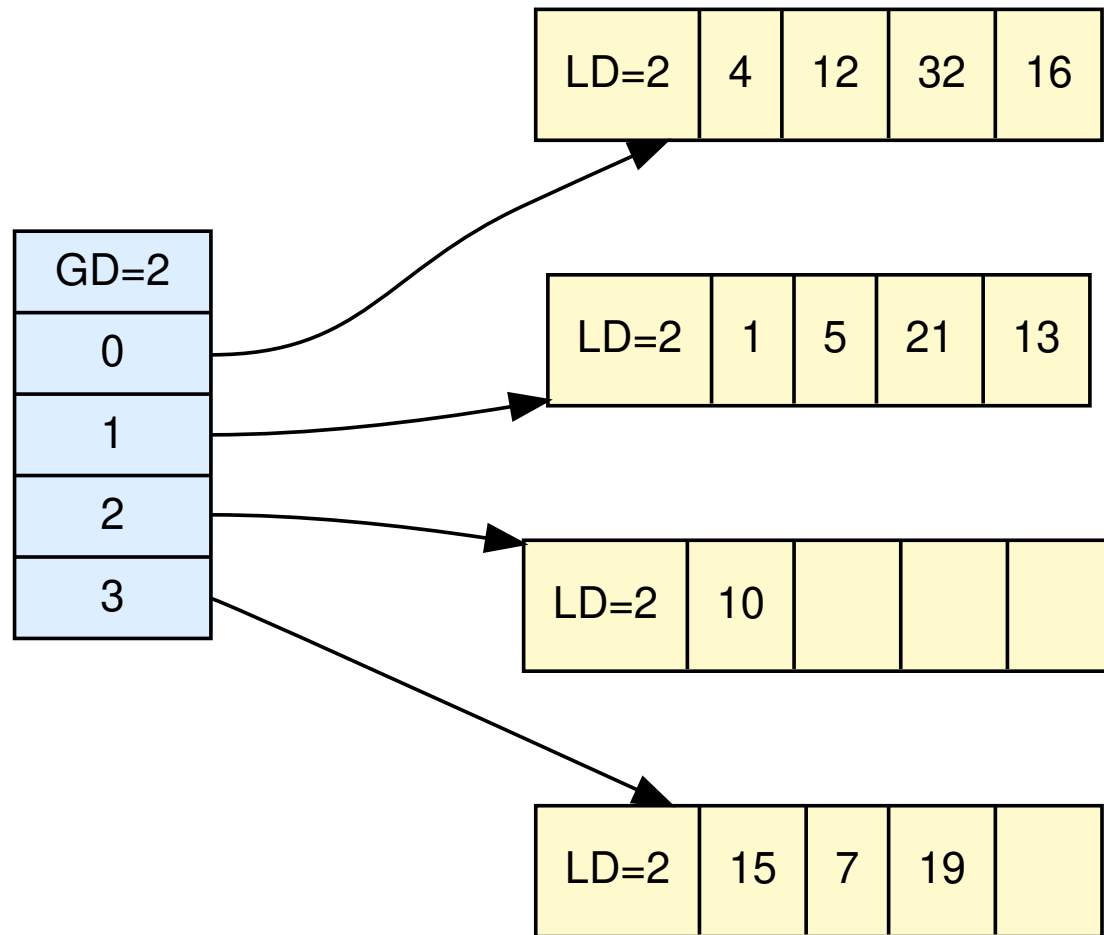
Add a 00010. No room. Directory doubles. Bucket splits.
(Doubling the table is annoying, need to stretch it)

Assume a bucket capacity of **4 keys**.

Directory shows $h(k) \% 2^{GD}$.

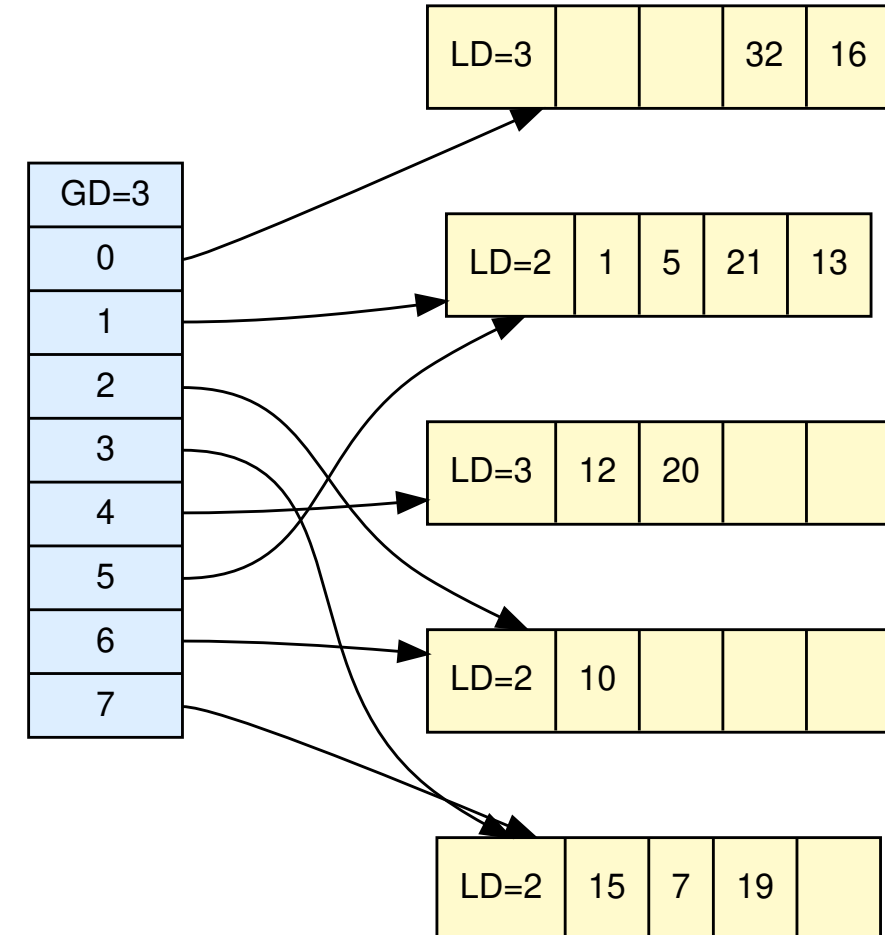
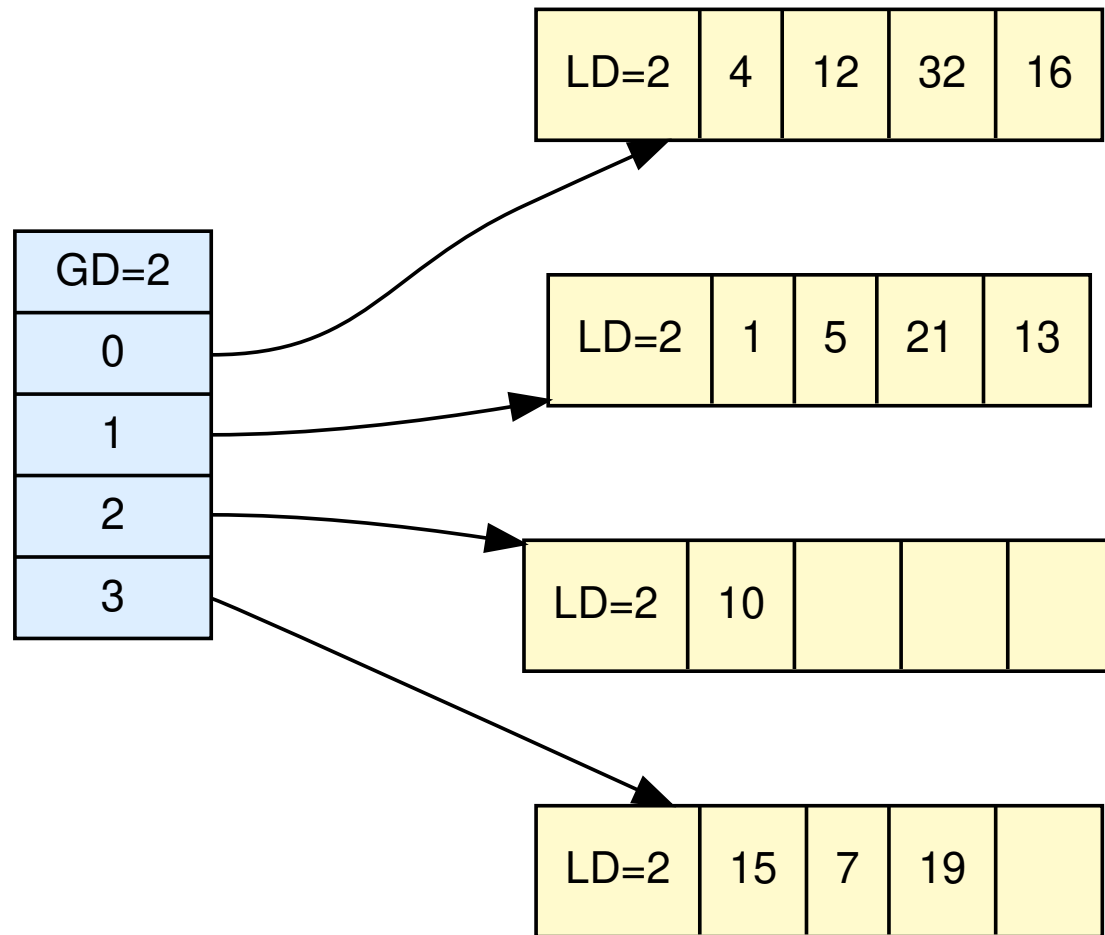


Add a 13. $13 \% 2^2 = 1$.



Add a 20. $20 \% 2^2 = 0$. No room. Directory doubles, bucket splits.

Add a 13. $13 \% 2^2 = 1$.



Add a 20. $20 \% 2^2 = 0$. No room. Directory doubles, bucket splits.

(Expanding directory is easier, just copy)

Sources on extendible hashing in databases are remarkably vague on where the extendible hashing directory lives. Here are some suggestions. Each one of these strategies is problematic for different reasons. Discuss for each of the following what the key problem is. Click on the choice that is least problematic.

- A. In memory, outside of the buffer pool
- B. On a sequential set of blocks, and thus in the buffer pool when the necessary blocks happen to be there
- C. On a randomly allocated set of blocks, and thus in the buffer pool when the necessary blocks happen to be there
- D. None of the above: if you pick this, you have to volunteer (when I ask) to tell us what would be better

GD=3
0
1
2
3
4
5
6
7

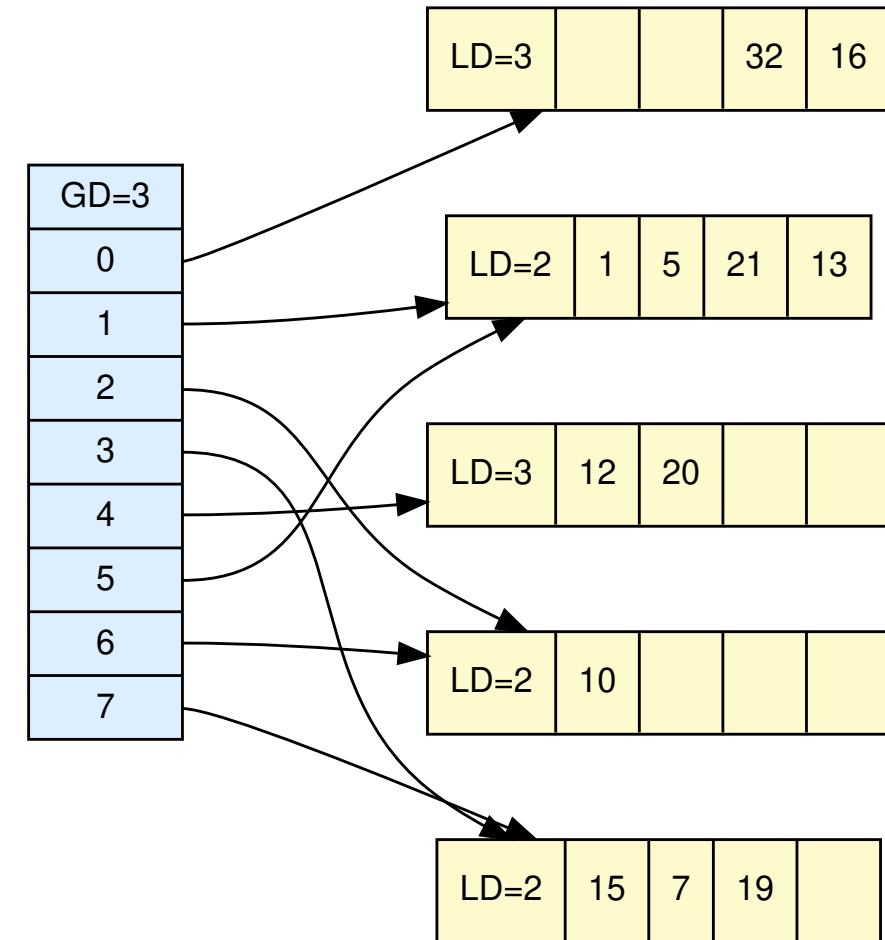
Where is directory stored? What if too big for buffer pool?

- each entry small: just a pageid each
- typical directory page (4k) can store ~ 1000
- if can store it sequentially, can then just do arithmetic to figure out which page id you need
- worst case build a sparse tree(?) index to sequential sections

How to delete?

- If a bucket becomes empty, merge with sibling
- If all elements in directory share pointer with sibling, cut dir in half

Duplicates: use overflow buckets



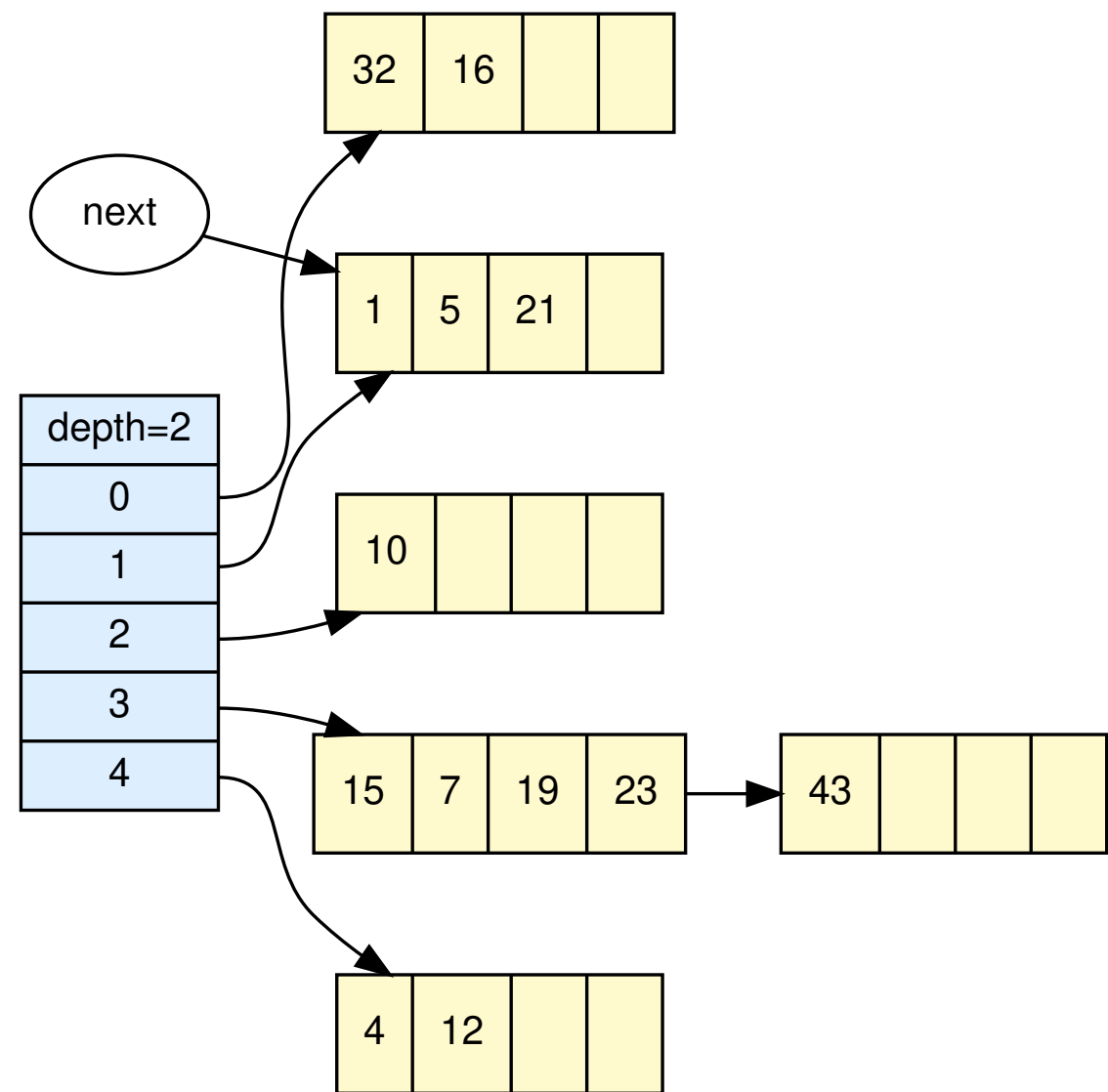
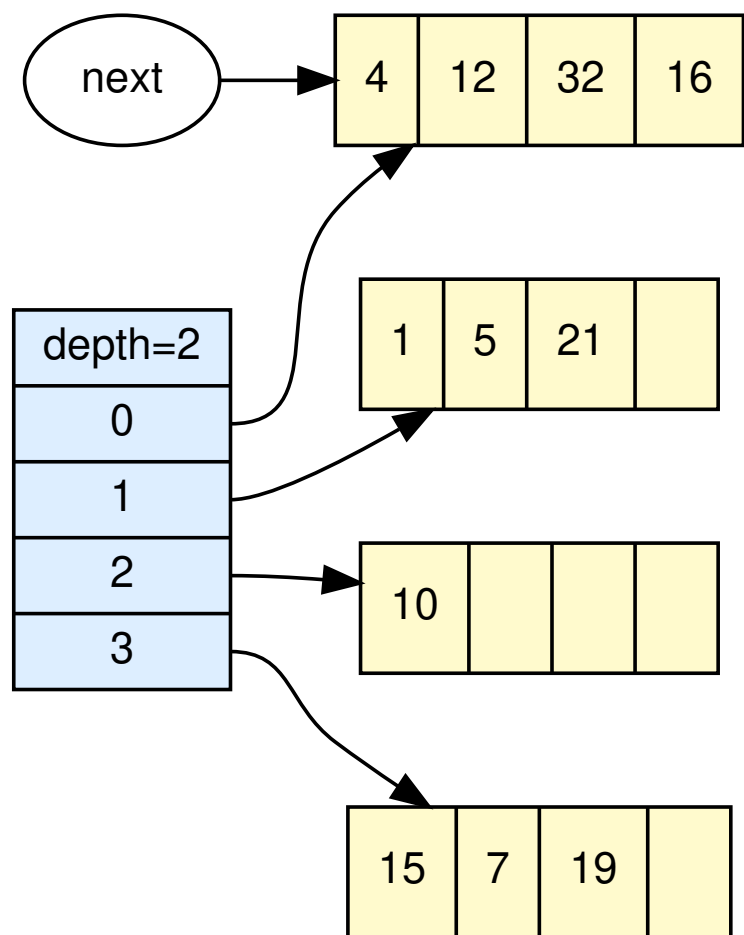
A B+ tree leaf node contains search keys and either the records themselves (in a primary index) or RIDs (in a secondary index). What's the difference internally between a so-called extendable hashing “bucket” and a B+ tree leaf node?

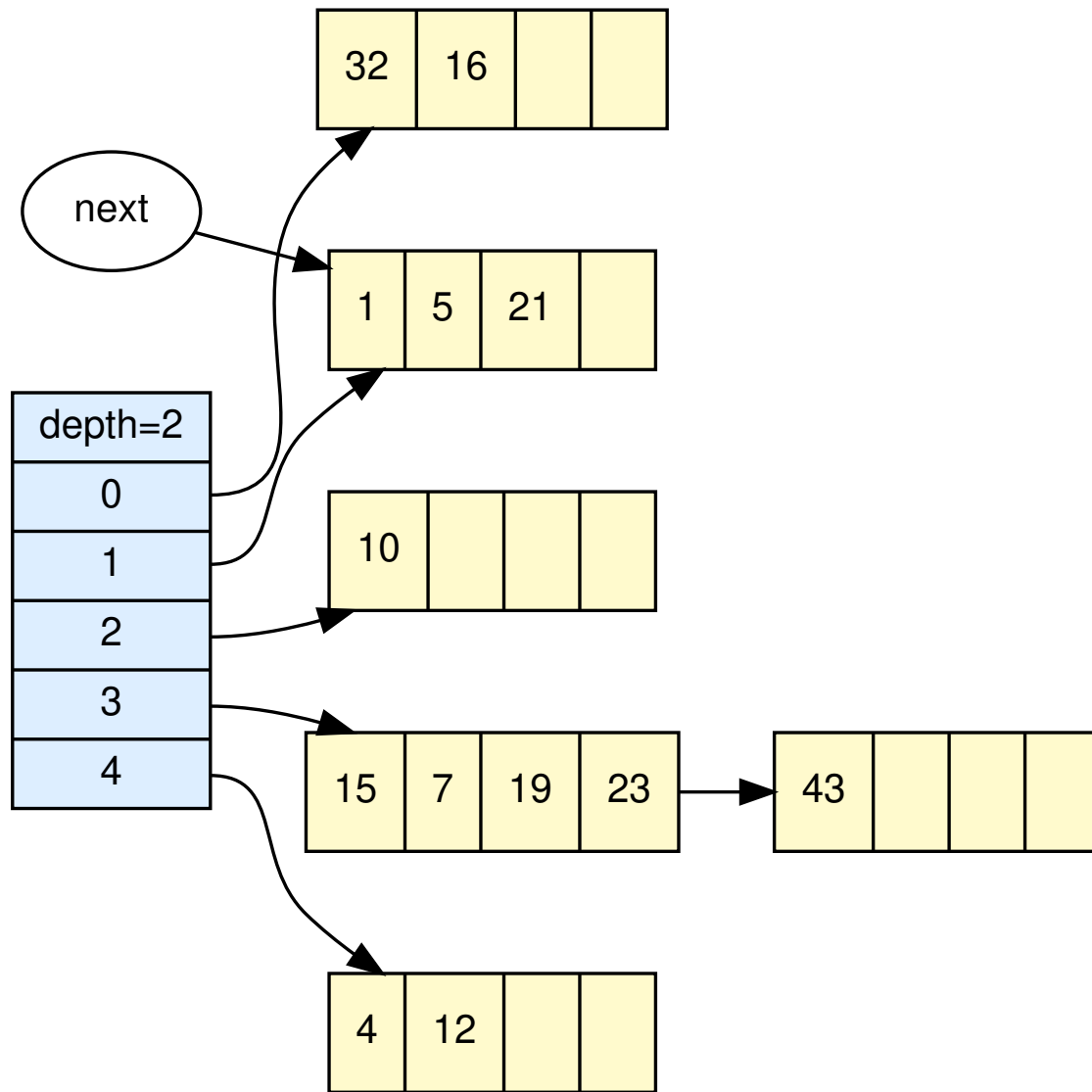
- A. A hash bucket adds an extra level; they point, in turn, to blocks that function like B+ tree leaf nodes
- B. Hash buckets contain n search keys and n pointers, whereas B+ tree leaves contain n search keys and $n + 1$ pointers
- C. There is no difference
- D. None of the above

Extendable hashing uses both a directory of pointers, and buckets. When do you double the directory?

- A. When the directory gets full
- B. When a bucket gets full
- C. Both A and B
- D. Neither A nor B

Linear hashing: split round-robin





To do a lookup: try $h(key) \% 2^{depth}$. If comes above “next”, use $h(key) \% 2^{depth+1}$. Increment depth when next cycles around.

- Duplicates are easy
- If can allocate first page of each in sequence, no directory needed, saves disk read